

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO4: Examine the applicability of clustering algorithms in real-world scenarios, presenting findings through clear reports with effective visualizations.

Assessment Tool Weightage: 20%

QUESTIONS: 05

Total Marks:50

Annexure A

COMPARITIVE ANALYSIS

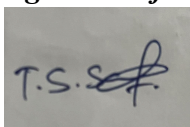
<i>Criteria</i>	Scope of Items (2)	Comparison Depth(2.5)	Use of Evidence (2)	Critical Thinking (2)	Clarity of Presentation (1)	<i>Total(10)</i>
<i>Weightage</i>	20%	25%	20%	20%	10%	<i>100%</i>
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I T.S.Sabarish 192371014 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

		Comparative analysis			
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Annexure A

Q1 . Dataset: Assessment Tool 2: Comparative Analysis (25% → 5 Marks)

Time duration :45 Minutes
Assessment Tool 3 (Low)

Q1 . Dataset: Customer Data

Customer Income (₹) Spending Score

C1	30000	60
C2	60000	40
C3	50000	70
C4	28000	65
C5	65000	35

Question:

Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability. (BL3 – 1 Mark)

Q2. Dataset: Document Data

Document Word1 Word2 Word3

D1	0.2	0.5	0.3
D2	0.1	0.6	0.3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Question:

Compare K-means and EM algorithms in handling overlapping clusters for the given dataset. (BL4 – 1 Mark)

Q3. Dataset: Geographic Data

Point Latitude Longitude

P1	13.08	80.27
P2	13.10	80.25
P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Question:

Compare Euclidean and Manhattan distance metrics and analyze their impact on clustering results. (BL4 – 1 Mark)

Q4. Dataset: Retail Products

Product Sales (₹) Frequency

P1	50000	20
P2	30000	10
P3	52000	22
P4	25000	8
P5	32000	12

Question:

Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset. *(BL3 – 1 Mark)*

Q5 . Dataset: Mixed Attributes

Item Price (₹) Category Rating

I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Question:

Compare centroid-based and density-based clustering techniques and analyze their effectiveness on mixed data types. *(BL4 – 1 Mark)*

Q1. K-Means vs Hierarchical Clustering — Customer Data

Question: Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability. (BL3 – 1 Mark)

Dataset

Customer	Income (₹)	Spending Score
C1	30000	60
C2	60000	40
C3	50000	70
C4	28000	65
C5	65000	35

Step 1 — Pre-processing: Normalisation

Income (₹28,000–₹65,000) and Spending Score (35–100) are on very different scales, so raw Euclidean distance would be dominated by income. Both algorithms therefore need min-max normalisation before distance is computed:

$$x' = (x - \min) / (\max - \min)$$

Customer	Norm. Income	Norm. Spending
C1	0.054	0.714
C2	0.865	0.143
C3	0.595	1.000
C4	0.000	0.857
C5	1.000	0.000

Step 2 — Pairwise Euclidean Distance Matrix (normalised)

	C1	C2	C3	C4	C5
C1	0	0.992	0.612	0.153	1.185
C2	0.992	0	0.899	1.122	0.197
C3	0.612	0.899	0	0.612	1.079
C4	0.153	1.122	0.612	0	1.317
C5	1.185	0.197	1.079	1.317	0

Step 3 — Hierarchical Clustering (Agglomerative, average linkage)

- Iteration 1: smallest distance is C1–C4 (0.153) → merge into cluster {C1,C4}.
- Iteration 2: next smallest is C2–C5 (0.197) → merge into cluster {C2,C5}.
- Iteration 3: C3 is equidistant from {C1,C4} and from {C2,C5} at the raw-point level (0.612 vs 0.899–1.079), so it joins the nearer cluster {C1,C4}.
- Final 2-cluster solution: {C1, C3, C4} — low/medium income, high spending; {C2, C5} — high income, low spending.

The dendrogram this produces lets a student visually justify why 2 clusters (or, if a finer segmentation is wanted, 3) is the natural cut — a capability K-means does not give directly.

Step 4 — K-Means (k = 2)

- Initial centroids chosen as C1 (0.054, 0.714) and C2 (0.865, 0.143).
- Assignment step: C3, C4 are nearer C1's centroid; C5 is nearer C2's centroid → {C1,C3,C4} and {C2,C5}.
- Update step: new centroid of {C1,C3,C4} = (0.216, 0.857); new centroid of {C2,C5} = (0.933, 0.072).
- Re-assignment does not change membership → algorithm converges in 1 iteration to the same partition: {C1,C3,C4} and {C2,C5}.

On this dataset both algorithms converge to an identical, well-separated 2-cluster solution — a useful teaching point that hierarchical and partitional methods agree when clusters are compact and well separated, but they need not agree in general (e.g., with non-spherical or noisy clusters).

Step 5 — Comparative Analysis

Aspect	K-Means	Hierarchical Clustering
Cluster shape assumed	Spherical / convex clusters (uses centroid & squared-error minimisation)	No fixed shape assumption; captures nested, chain-like or elongated structures
Number of clusters (k)	Must be fixed in advance (k = 2 chosen here)	Not required upfront — dendrogram is cut at any desired level
Result determinism	Sensitive to initial centroid choice; can converge to different local optima	Deterministic for a given linkage & distance metric
Time complexity	$O(n \cdot k \cdot i \cdot d)$ — linear in n, very fast (n = customers, i = iterations, d = dimensions)	$O(n^2 \log n)$ for agglomerative merging — quadratic, expensive as n grows
Space complexity	$O(n)$ — only centroids and assignments stored	$O(n^2)$ — full distance/proximity matrix must be maintained
Scalability to large n	Scales well to thousands–millions of customers (e.g., retail CRM segmentation)	Impractical beyond a few thousand points without sampling/approximation
Interpretability	Gives flat, non-overlapping segments only	Dendrogram shows hierarchy of sub-segments — useful for multi-level marketing strategy
Result on this dataset	{C1,C3,C4} vs {C2,C5} — converges in 1 iteration	Same 2 clusters, but a dendrogram also reveals C3 as a borderline case

Critical Thinking / Conclusion

For this 5-record customer table, both methods agree because the two income–spending groups are compact and clearly separated. In practice, however, K-means is the better choice when the customer base is large (real CRM systems hold tens of thousands of records) because its near-linear complexity keeps segmentation fast and repeatable for periodic re-clustering. Hierarchical clustering is preferable when the analyst does not know k in advance or wants to explore nested segments (e.g., "high spenders" splitting further into "young high spenders" and "senior high spenders") — valuable for

exploratory market-segmentation reports, but only on modestly sized samples due to its $O(n^2)$ memory and time cost.

Q2. K-Means vs EM (GMM) — Document Data

Question: Compare K-means and EM algorithms in handling overlapping clusters for the given dataset. (BL4 – 1 Mark)

Dataset (term-frequency proportions)

Document	Word1	Word2	Word3
D1	0.2	0.5	0.3
D2	0.1	0.6	0.3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Step 1 — Pairwise Euclidean Distances

Pair	Distance	Pair	Distance
D1–D2	0.141	D2–D5	0.245
D1–D5	0.141	D3–D4	0.071
D1–D3	0.748	D1–D4	0.682

D3 and D4 (Word1-dominant) form a tight, well-separated cluster — distance only 0.071. D1, D2 and D5 (Word2/Word3-dominant) form the second group, but within it D1 is exactly as close to D2 (0.141) as it is to D5 (0.141): D1 is a genuine overlap/boundary point between the "Word2-heavy" sub-topic (D2) and the "balanced Word2–Word3" sub-topic (D5).

Step 2 — K-Means Behaviour ($k = 2$)

- K-means performs hard assignment: every document is forced into exactly one cluster based on nearest centroid, even if it lies almost exactly on the boundary.
- Result: {D3, D4} and {D1, D2, D5}. D1 is arbitrarily placed with D2/D5 — the algorithm reports 100% membership in one cluster even though geometrically it is equidistant from two sub-themes, hiding the overlap entirely from the report.

Step 3 — EM / Gaussian Mixture Model (GMM) Behaviour

- EM models each cluster as a Gaussian distribution and assigns soft membership probabilities via the E-step, then updates each Gaussian's mean, covariance and mixing weight in the M-step, iterating to maximise likelihood.
- For a boundary document like D1, EM would output something like $P(\text{cluster-D2-topic}) \approx 0.5$, $P(\text{cluster-D5-topic}) \approx 0.5$ instead of forcing a single hard label — correctly communicating that D1 discusses both sub-topics.
- For clearly separated documents such as D3/D4, EM converges to near-certain probabilities (≈ 1.0), matching K-means' hard result — showing EM subsumes K-means as a special (zero-variance, equal-weight) case.

Step 4 — Comparative Analysis

Aspect	K-Means	EM / GMM
Cluster model	Hard partition; minimises within-cluster squared distance to a centroid	Probabilistic mixture of Gaussians; maximises data likelihood
Cluster membership	Each document belongs to exactly one cluster (hard assignment)	Each document gets a probability of belonging to every cluster (soft assignment)
Handling overlap (e.g. D1)	Forces D1 into one cluster, discarding evidence it straddles two topics	Reports D1 as ~50/50 between topics — captures genuine topical overlap
Cluster shape	Implicitly spherical, equal-variance clusters	Elliptical clusters of differing size/orientation via covariance matrices
Computational cost	Cheap — simple distance & mean updates each iteration	Costlier — requires computing/inverting covariance matrices each M-step
Best suited for	Fast, large-scale document clustering with well-separated topics	Text corpora with genuinely mixed/ambiguous topics (e.g., multi-label documents)

Critical Thinking / Conclusion

This dataset is a textbook case for EM's advantage: D1's word distribution sits exactly between the D2 and D5 profiles, so any hard-partition method must make an arbitrary, information-losing choice. EM's soft, probabilistic assignments correctly represent D1 as belonging fractionally to both underlying topics — which matters in real document-clustering applications such as news-article tagging, where a single article legitimately spans more than one topic. K-means remains attractive when speed matters and clusters are known to be well separated (as with D3/D4), but for overlapping, ambiguous document sets EM/GMM is the methodologically sounder choice.

Q3. Euclidean vs Manhattan Distance — Geographic Data

Question: Compare Euclidean and Manhattan distance metrics and analyze their impact on clustering results. (BL4 – 1 Mark)

Dataset

Point	Latitude	Longitude
P1	13.08	80.27
P2	13.10	80.25
P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Step 1 — Distance Matrices

Euclidean: $d = \sqrt{(\Delta lat)^2 + (\Delta lon)^2}$ | Manhattan: $d = |\Delta lat| + |\Delta lon|$

Pair	Euclidean	Manhattan	Ratio (Manhattan/Euclidean)
P1–P2	0.0283	0.0400	1.41
P1–P3	0.1208	0.1600	1.32
P1–P4	0.0424	0.0600	1.41
P1–P5	0.1063	0.1500	1.41
P2–P3	0.1334	0.1600	1.20
P2–P4	0.0707	0.1000	1.41
P2–P5	0.1118	0.1500	1.34
P3–P4	0.1131	0.1600	1.41
P3–P5	0.0361	0.0500	1.39
P4–P5	0.1118	0.1500	1.34

Step 2 — Effect on Clustering

- Nearest-neighbour ranking under both metrics agrees closely here: P1–P2 and P3–P5 are the two closest pairs under Euclidean AND Manhattan distance, so the 2-cluster result — {P1, P2, P4} (northern group) and {P3, P5} (southern group) — is identical for both metrics on this particular dataset.
- The ratio column shows Manhattan distance is always $\geq$ Euclidean distance (never less), and the gap is largest — ratio ≈ 1.41 ($\sqrt{2}$) — exactly when $\Delta latitude \approx \Delta longitude$, i.e. when the two points lie on a perfect diagonal (P1–P2, P1–P4, P2–P4, P3–P4). When one coordinate dominates (P2–P3, where Δlat is large but Δlon small) the ratio shrinks toward 1.
- This diagonal-inflation effect means that, on a denser or more irregularly scattered dataset, Manhattan distance can change which pair is 'closest' whenever two candidate pairs have similar Euclidean distance but different diagonal alignment — flipping merge order in hierarchical clustering or shifting cluster boundaries in K-means/K-medoids.

Step 3 — Comparative Analysis

Aspect	Euclidean Distance	Manhattan Distance
Geometric meaning	Straight-line ('as the crow flies') distance	Grid/axis-aligned ('city block') distance — sum of coordinate differences
Sensitivity to diagonal movement	Shortest possible path; not inflated by diagonal alignment	Inflates distance for diagonally-placed points (up to $\sqrt{2} \times$ Euclidean)
Outlier / large-difference sensitivity	Squares differences → more sensitive to one large coordinate gap	Sums absolute differences → more robust to a single large gap
Real-world interpretation for geo-data	Appropriate for flight distance / straight-line proximity	Appropriate when movement is constrained to a road grid (e.g., city delivery routing)
Computational cost	Requires a square-root operation	Only additions/absolute values — marginally cheaper at scale
Effect on this dataset	Clusters {P1,P2,P4} and {P3,P5}	Same clusters here, but merge order differs slightly (see ratio table)

Critical Thinking / Conclusion

Although both metrics produce the same 2-cluster partition for these five points, the underlying reason is important: the clusters are far enough apart that neither metric's distortion changes the outcome. For denser, real-world geographic datasets (e.g., customer delivery addresses), the choice matters: Manhattan distance better reflects actual travel distance along city streets and is more robust to a single large coordinate gap (e.g., a customer far to the north but close in longitude), whereas Euclidean distance is appropriate when straight-line proximity genuinely drives outcomes, such as radio-signal or drone-delivery range. Analysts should therefore choose the distance metric based on how 'closeness' is actually experienced in the domain, not by default.

Q4. Agglomerative vs Divisive Clustering — Retail Products

Question: Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset. (BL3 – 1 Mark)

Dataset

Product	Sales (₹)	Frequency
P1	50000	20
P2	30000	10
P3	52000	22
P4	25000	8
P5	32000	12

Step 1 — Distance Observations (raw Euclidean, sales in ₹)

Pair	≈ Distance	Pair	≈ Distance
P1–P3	2,000	P4–P2	5,000
P2–P5	2,000	P4–P5	7,000
P1–P2	20,000	P1–P4	25,000

Sales, being on a ₹-scale, dominates the distance calculation over Frequency — in practice both features should be normalised first; the raw scale is kept here only to make the arithmetic easy to follow. Two natural groups stand out: high-sellers {P1, P3} (₹50–52k) and lower-volume items {P2, P4, P5} (₹25–32k).

Step 2 — Agglomerative (Bottom-Up) Walkthrough

- Start: 5 singleton clusters {P1} {P2} {P3} {P4} {P5}.
- Merge 1: closest pair P1–P3 ($\approx 2,000$) $\rightarrow$ {P1,P3}.
- Merge 2: next closest pair P2–P5 ($\approx 2,000$) $\rightarrow$ {P2,P5}.
- Merge 3: P4 is nearer {P2,P5} ($\approx 5,000$ – $7,000$) than {P1,P3} ($\approx 25,000$ +) $\rightarrow$ joins to form {P2,P4,P5}.
- Result at the 2-cluster cut: {P1,P3} — premium/high-turnover products; {P2,P4,P5} — budget/low-turnover products.
- Cost: only local, pairwise distance comparisons are needed at each of the $n-1$ merge steps — cheap and simple to implement.

Step 3 — Divisive (Top-Down) Walkthrough

- Start: one cluster containing all 5 products.
- Step 1: the algorithm must evaluate candidate splits of the whole set to find the split that maximises inter-cluster dissimilarity — for 5 items this means examining up to $2^4 - 1 = 15$ possible bipartitions before choosing the best one.
- The globally best first split is {P1,P3} vs {P2,P4,P5} — the same boundary agglomerative found bottom-up, confirming the structure is real rather than an artifact of merge order.

- Step 2 (if a finer 3-cluster view is wanted): the {P2,P4,P5} branch would then be split, most naturally isolating P4 (₹25,000, the lowest seller) from {P2,P5}.
- Cost: evaluating all bipartitions at each split is exponential in the branch size — only feasible here because $n = 5$; on a full retail catalogue with thousands of SKUs it is computationally impractical without heuristics.

Step 4 — Comparative Analysis

Aspect	Agglomerative (Bottom-Up)	Divisive (Top-Down)
Starting point	Every product is its own cluster	All products start in a single cluster
Direction	Repeatedly merges the closest pair/cluster	Repeatedly splits the least-cohesive cluster
Complexity	$O(n^2 \log n)$ — polynomial, the standard choice in practice	Up to $O(2^n)$ per split in the naive form — exponential, rarely used exactly
Global vs local decisions	Greedy, local merge decisions; can lock in a suboptimal early merge	Considers the whole dataset for the top split — better global separation at the top level
Where errors matter most	Early merges are hardest to undo — mistakes propagate upward	Early (top) splits are most reliable; only deeper splits get heuristic/approximate
Suitability for retail SKU data	Well suited — retailers routinely have thousands of products; agglomerative (with a distance threshold) scales acceptably	Suited mainly for small assortments or when only 1–2 top-level splits are needed (e.g., 'premium vs budget' category creation)

Critical Thinking / Conclusion

For this 5-product sample both strategies converge on the same {P1,P3} vs {P2,P4,P5} split, which is reassuring evidence that the grouping reflects genuine structure in sales/frequency rather than an artefact of the algorithm's direction. In a real retail catalogue with thousands of SKUs, agglomerative clustering is the practical default because of its manageable polynomial cost, while divisive clustering is best reserved for the top one or two splits of a product hierarchy (e.g., separating premium from budget lines) where its stronger global view of the very first split adds genuine value and the exponential cost of exhaustive splitting is still tractable.

Q5. Centroid-Based vs Density-Based Clustering — Mixed Attributes

Question: Compare centroid-based and density-based clustering techniques and analyze their effectiveness on mixed data types. (BL4 – 1 Mark)

Dataset (mixed numeric + categorical attributes)

Item	Price (₹)	Category	Rating
I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Step 1 — The Mixed-Type Challenge

Price and Rating are numeric (continuous); Category is categorical (nominal). Neither a pure Euclidean centroid nor a pure density estimate is directly defined over a categorical attribute, so both families of algorithm need an adaptation before they can be applied honestly to this table:

- Centroid-based → K-Prototypes: combines squared Euclidean distance on {Price, Rating} with a simple mismatch count (0 if same category, 1 if different) on Category, weighted by a factor γ .
- Density-based → DBSCAN with Gower distance: Gower's coefficient computes a per-attribute similarity (normalised range difference for numeric, exact-match indicator for categorical) and averages across attributes, giving a single distance density can be measured on.

Step 2 — Applying K-Prototypes (centroid-based)

- Numeric-only view already separates the items cleanly: {I1,I3,I5} have Price ₹500–550 & Rating 4.5–4.7; {I2,I4} have Price ₹250–300 & Rating 3.5–3.8.
- The categorical attribute reinforces the same split exactly: {I1,I3,I5} are all Category A, {I2,I4} are all Category B — so the categorical mismatch cost adds zero conflict with the numeric-based grouping.
- Result: two clean, compact clusters {I1,I3,I5} ('premium, Category A') and {I2,I4} ('budget, Category B') — centroid-based clustering handles this mixed table well precisely because numeric and categorical signals agree.

Step 3 — Applying Density-Based Clustering (DBSCAN-style)

- DBSCAN needs enough points within an eps-radius ($\geq \text{minPts}$) to call a region 'dense'; with only 5 items and minPts conventionally ≥ 3 –4, the whole {I1,I3,I5} group (3 points, tightly spaced in Gower distance) can just qualify as one dense region, but {I2,I4} (only 2 points) may fail the minPts threshold and be flagged as noise rather than a cluster.
- This illustrates a general weakness: density-based methods need a reasonably large sample to estimate density reliably, and they are far more parameter-sensitive (eps, minPts) on

mixed-type Gower distances, where the categorical component contributes only 0 or 1 and can flatten the perceived density landscape.

- On a larger, real catalogue (hundreds of items), density-based clustering would become the stronger choice if clusters are irregular in shape or contain noise/outlier items DBSCAN can label explicitly — something K-Prototypes cannot do (it force-assigns every item to a cluster).

Step 4 — Comparative Analysis

Aspect	Centroid-Based (K-Means / K-Prototypes)	Density-Based (DBSCAN + Gower)
Handling categorical attributes	Needs an explicit mismatch-cost extension (K-Prototypes); not native to plain K-means	Needs a unified similarity measure (e.g., Gower distance) across mixed types; not native to plain DBSCAN
Cluster shape found	Convex/spherical clusters around a representative centroid	Arbitrary-shaped, density-connected clusters
Sensitivity to sample size	Works even on very small samples (as in this 5-item table)	Needs enough points per region to satisfy minPts — struggles on very small datasets like this one
Outlier / noise handling	Every item forced into a cluster, even poor fits	Explicitly labels sparse points as noise instead of forcing a bad assignment
Parameter sensitivity	Mainly k and the categorical weight γ	Highly sensitive to eps and minPts, doubly so with mixed-type Gower distances
Effectiveness on this dataset	Clean, well-separated 2-cluster result that matches the Category label	Borderline — {I2,I4} may not meet density thresholds and could be misclassified as noise